Universidade Estadual da Paraíba UEPB
Centro de Ciência e Tecnologia
Departamento de Computação

Antonio da Silva Lins
Artur Oliveira Praxedes
Marcella Viana da Silva Lins

ANÁLISE COMPARATIVA DE ESTRATÉGIAS DE BUSCA APLICADAS AO PROBLEMA DE ESCALONAMENTO DE TAREFAS

Campina Grande - PB
2026

SUMÁRIO

1. INTRODUÇÃO.....3
1.1. Contextualização do Problema..... 3
1.2. Objetivos..... 3
1.2.1. Geral.....3
1.2.2. Específicos..... 3
2. METODOLOGIA E IMPLEMENTAÇÃO..... 4
2.1. Estrutura do Sistema Desenvolvido..... 4
2.2. Formato de Dados de Entrada e Saída.....4
2.3. Métricas de Avaliação..... 6
3. ALGORITMOS IMPLEMENTADOS.....6
3.1. Backtracking.....6
3.1.1. Análise de Corretude.....7
3.1.2. Análise de Desempenho..... 7
3.1.2.1. Espacial..... 9
3.1.2.2. Temporal.....9
3.2. Branch and Bound.....9
3.2.1. Análise de Corretude..... 11
3.2.2. Análise de Desempenho..... 12
3.2.2.1. Espacial..... 13
3.2.2.2. Temporal.....13
3.3. Greedy.....14
3.3.1. Análise de Corretude.....14
3.3.2. Análise de Desempenho..... 16
3.3.2.1. Espacial..... 16
3.3.2.2. Temporal.....17
3.4. Dynamic Programming.....17
3.4.1. Análise de Corretude.....18
3.4.2. Análise de Desempenho..... 19
3.4.2.1. Temporal.....19
3.4.2.2. Espacial..... 19
4. EXPERIMENTOS E RESULTADOS..... 20
4.1. Instâncias de Teste.....20
4.2. Resultados Obtidos.....21
4.2.1. Configurações do Ambiente..... 21
4.2.2. Análise dos Resultados.....21
5. ANÁLISE E DISCUSSÃO.....24
5.1. Comparação das Estratégias.....24
6. CONCLUSÃO..... 25
7. REFERÊNCIAS..... 27

1.INTRODUÇÃO

1.1. Contextualização do Problema

O problema de escalonamento de tarefas consiste em determinar uma ordem de execução para um conjunto de tarefas de forma a otimizar um critério de desempenho. Cada tarefa possui características específicas, como tempo de processamento, prazo para conclusão e um valor associado, que representa o ganho obtido ao executar a tarefa dentro das condições estabelecidas. A sequência em que essas tarefas são executadas influencia diretamente no resultado final do sistema, especialmente considerando possíveis penalidades por atraso ou perda de valor.

Nesse contexto, uma solução corresponde a uma permutação das tarefas disponíveis, indicando a ordem em que serão executadas. O objetivo é encontrar a sequência que maximize a função objetivo definida, levando em conta os valores das tarefas e suas restrições. Entretanto, à medida que o número de tarefas aumenta, o número de possíveis sequências cresce rapidamente, tornando a busca pela melhor solução computacionalmente desafiadora.

1.2. Objetivos

1.2.1. Geral

Implementar e analisar estratégias para resolver o problema de escalonamento de tarefas, avaliando o desempenho das soluções obtidas.

1.2.2. Específicos

- Modelar o problema de escalonamento de tarefas e representar suas possíveis soluções.
- Implementar estratégias para encontrar a ordem de execução das tarefas.
- Realizar experimentos e comparar o desempenho das estratégias com base em métricas como tempo de execução, uso de memória e espaço de busca explorado.

2. METODOLOGIA E IMPLEMENTAÇÃO

2.1. Estrutura do Sistema Desenvolvido

O projeto foi desenvolvido utilizando a linguagem de programação Java. Tal escolha se deu devido a abordagem de Orientação de Objetos, que facilita a organização e manipulação das estruturas. Esse paradigma permite uma melhor modularização do código, contribuindo para a implementação dos algoritmos e para a realização de testes durante o desenvolvimento.

Tabela 01: Estrutura do Sistema Desenvolvido

Componentes	Classes	Descrição
main	Main	Inicia e controla a execução do sistema.
model	Task e Solution	Representação das tarefas e das soluções do problema.
algorithms	Backtracking, BranchAndBound, Greedy e Dynamic Programming	Implementação das estratégias de busca utilizadas.
utils	InputGenerator; InstanceReader; ResultWriter; TimerUtils	Geração dos dados e salva em txt; Cria objetos a partir dos dados lidos; Salva resultados em arquivo txt; Mede o tempo de execução.
instances	—	Armazena as instâncias de entrada.
results	—	Armazena os resultados gerados.

Fonte: elaborada pelos autores

2.2. Formato de Dados de Entrada e Saída

Os dados de entrada e saída do sistema são armazenados em arquivos no formato .txt, escolhidos pela facilidade de manipulação utilizando as bibliotecas da linguagem Java. Essa organização será formatada conforme as tabelas abaixo:

Tabela 02: Formato dos arquivos de Entrada

Tipo de Arquivo	Exemplo	Descrição
Input	5 2 14 52 7 16 32 1 29 61 5 24 37 10 24 43 10 5 20 53 8 8 69 3 10 84 7 27 8 3 20 34 9 26 65 9 11 73 3 13 23 7 24 12 7 8 10	A primeira linha de cada instância indica o número de tarefas (n), e as n linhas seguintes contêm três números inteiros separados por espaço: tempo de processamento, prazo e valor da tarefa. Linhas em branco podem separar instâncias, mas são ignoradas pelo programa.

Fonte: elaborada pelos autores

Tabela 03: Formato dos arquivos de Saída

Tipo de Arquivo	Exemplo	Descrição
Output	Instancia 1 Backtracking 229 4 61 18 76 74 BranchAndBound 229 4 61 18 76 74 Greedy 229 4 74 61 76 18 DynamicProgramming 229 4 61 18 76 74	Cada instância do resultado começa com um cabeçalho indicando o número da instância. Em seguida, são apresentados os resultados do algoritmo Backtracking e do Branch and Bound, respectivamente. Para cada algoritmo, a primeira linha contém o nome do algoritmo, a segunda linha indica o valor total da solução encontrada, a terceira linha apresenta a quantidade de tarefas selecionadas na solução, e a quarta linha lista os valores das tarefas escolhidas, separados por espaço.

	----------	Após as informações de cada instância, uma linha com separadores (“----------”) é usada para diferenciar visualmente as instâncias.

Fonte: elaborada pelos autores

2.3. Métricas de Avaliação

Para comparar o desempenho dos algoritmos foram consideradas métricas de eficiência e qualidade das soluções. Entre elas estão o tempo de execução, o uso de memória, o espaço de busca explorado e as podas realizadas durante a execução.Também foi avaliada a escalabilidade, para verificar como os algoritmos se comportam com instâncias de tamanhos diferentes. Essas métricas permitem comparar de forma clara a eficiência e o consumo de recursos de cada técnica.

3.ALGORITMOS IMPLEMENTADOS

3.1. Backtracking

O Backtracking é uma técnica algorítmica utilizada para resolver problemas de busca e satisfação de restrições por meio da exploração sistemática das possíveis combinações de soluções. Nesse método, as soluções são construídas de forma incremental, avaliando-se a cada etapa se ainda atendem às restrições impostas pelo problema. Caso uma solução parcial viole alguma restrição, o algoritmo retrocede (backtrack), descartando esse caminho e explorando novas possibilidades. Uma característica importante desse algoritmo é a poda, que consiste em interromper antecipadamente a exploração de caminhos que não podem levar a uma solução válida, reduzindo significativamente o espaço de busca analisado e tornando o processo mais eficiente (DECHTER; FROST, 2002).

3.1.1. Análise de Corretude

A corretude do algoritmo de Backtracking baseia-se na exploração completa do espaço de soluções possíveis, garantindo que todas as combinações viáveis de tarefas sejam consideradas.

Para cada tarefa, o algoritmo toma duas decisões possíveis: incluir ou não incluir a tarefa na solução. Essas decisões são organizadas em uma árvore de busca binária, onde cada nível representa uma tarefa e cada ramo corresponde a uma escolha.

Invariante de laço:

A cada chamada recursiva do método backtrack, a lista currentTasks representa uma solução parcial válida, ou seja, todas as tarefas incluídas até o momento respeitam suas respectivas restrições de deadline.

Inicialização:

Antes da primeira chamada recursiva, a solução parcial é vazia, sendo, portanto, válida. O valor atual da solução é zero, o que está correto, já que nenhuma tarefa foi selecionada.

Manutenção:

A cada chamada recursiva, o algoritmo mantém a validade da solução parcial ao incluir uma tarefa somente se a soma do tempo atual com o tempo de processamento da tarefa não ultrapassa o seu deadline. Caso contrário, essa escolha é descartada, garantindo que apenas soluções válidas sejam exploradas.

Término:

Quando todas as tarefas foram consideradas (caso base), o algoritmo compara a solução atual com a melhor solução encontrada até o momento. Como todas as combinações possíveis são exploradas, a solução final armazenada em bestSolution corresponde a uma solução ótima do problema.

3.1.2. Análise de Desempenho

O algoritmo de Backtracking possui uma estrutura recursiva em que, para cada tarefa, existem duas decisões possíveis: incluir ou não incluir a tarefa na solução. Isso gera a seguinte recorrência:

$T(n) = 2T(n - 1) + O(1)$

Onde:

- $T(n)$ representa o tempo para resolver um problema com n tarefas;
- $2T(n - 1)$ representa as duas chamadas recursivas (incluir ou não incluir);
- $O(1)$ corresponde ao custo constante das operações realizadas em cada chamada (verificações e atualizações).

Expandindo a recorrência:

$T(n) = 2T(n - 1) + c$

$T(n - 1) = 2T(n - 2) + c$

Substituindo:

$T(n) = 2[2T(n - 2) + c] + c$

$T(n) = 2^2T(n - 2) + 2c + c$

Continuando o processo:

$T(n) = 2^3T(n - 3) + 2^2c + 2c + c$

Generalizando:

$T(n) = 2^k T(n - k) + (2^k - 1)c$

Quando k = n:

$T(n) = 2^n T(0) + (2^n - 1)c$

Como T(0) é constante:

$T(n) = O(2^n)$

Portanto, a complexidade temporal do algoritmo é exponencial.

Além da complexidade temporal, é importante analisar o uso de memória do algoritmo. Como o Backtracking utiliza chamadas recursivas, o consumo de memória está diretamente relacionado à profundidade da recursão.

No pior caso, a profundidade da pilha de chamadas é proporcional ao número de tarefas n, já que o algoritmo percorre uma tarefa por nível da árvore de decisão. Dessa forma, o espaço utilizado pela pilha de execução é O(n).

Além disso, o algoritmo mantém estruturas auxiliares, como a lista de tarefas atuais (currentTasks) e a melhor solução encontrada (bestSolution), que também podem armazenar até n elementos. No entanto, essas estruturas não crescem exponencialmente, pois apenas uma solução parcial é mantida por vez.

Portanto, a complexidade espacial do algoritmo de Backtracking é O(n), sendo linear em relação ao número de tarefas.

3.1.2.1. Espacial

A complexidade espacial do algoritmo de Backtracking é O(n), onde n representa o número de tarefas.

Isso ocorre porque a profundidade máxima da recursão é n, já que o algoritmo percorre as tarefas sequencialmente. Além disso, a lista currentTasks, que armazena a solução parcial, pode conter no máximo n tarefas.

O algoritmo não armazena todas as soluções possíveis, apenas a solução atual e a melhor solução encontrada, o que mantém o consumo de memória linear.

3.1.2.2. Temporal

A complexidade temporal do algoritmo de Backtracking, no pior caso, é O(2^n), onde n é o número de tarefas.

Isso ocorre porque, para cada tarefa, existem duas escolhas possíveis (incluir ou não incluir), resultando em uma árvore de busca com até 2^n nós.

Embora o algoritmo utilize uma forma simples de poda ao verificar os deadlines, essa poda não elimina significativamente o número de combinações exploradas no pior caso. Dessa forma, o comportamento permanece exponencial.

Na prática, o desempenho pode ser melhor dependendo das instâncias de entrada, especialmente quando muitas combinações são descartadas rapidamente por violarem as restrições de deadline. Ainda assim, o crescimento exponencial domina à medida que o número de tarefas aumenta.

3.2. Branch and Bound

A técnica de projeto de algoritmos Branch and Bound é voltada e comumente utilizada para problemas de otimização, tendo como principal objetivo encontrar a melhor solução, mínima ou máxima, em um conjunto de soluções viáveis. É visto como uma adaptação e evolução do backtracking, já que enquanto o backtracking explora todas as combinações possíveis, o Branch and Bound usa bounds (limites) para evitar a exploração de partes da árvore de busca que não conseguem gerar soluções superiores do que a já encontrada pelo algoritmo. Resumindo, a técnica funciona com a divisão de dois conceitos principais: a melhor solução atual, denominada “best”, e uma estimativa de limite inferior ou superior para cada nó da

árvore de decisões, denominado “bound”, que permite a poda de ramos desinteressantes para a solução ótima.

O funcionamento do algoritmo consiste em: construir uma árvore de estados onde cada nó representa uma solução parcial. A cada branch (passo), o algoritmo decide expandir essa solução incluindo novas escolhas restantes. Antes de continuar com a adição de novas escolhas na exploração, é calculado um limite, denominado “upper bound”, que representa o melhor valor que ainda pode ser obtido através daquele nó. Se esse limite não for melhor que a melhor solução já encontrada, o ramo sofre um bound, ou seja, é podado já não compensa explorá-lo mais. Caso contrário, o algoritmo continua a exploração até chegar em soluções completas, e sempre atualizando a melhor solução ao encontrar uma mais valiosa.

Para o problema do escalonamento de tarefas, a implementação do Branch and Bound foi adaptado para o tratamento das deadlines, onde cada tarefa (task) possui tempo de processamento, valor (lucro) e prazo. O objetivo é identificar um subconjunto de tarefas que respeite os deadlines das tarefas e maximize o valor total. Inicialmente, na implementação desse algoritmo específico, as tarefas são ordenadas por deadline (prazo), o que ajuda a respeitar as restrições e otimizar minimamente já de início o código. A árvore de busca é construída considerando que, para cada tarefa, existem duas possibilidades: incluir ou não incluir a tarefa. Quando uma tarefa (task) é incluída, o algoritmo verifica se ainda é possível realizar sua execução dentro do prazo (deadline).

A função de poda (bound) é moldada com base em um upper bound simples, esse é calculado pela soma do valor atual com o valor de todas as tarefas restantes, esse representado pela variável calculateRemainingValue. Esse valor irá representar o melhor cenário atingível a partir daquele ponto. Se esse limite for menor ou igual ao melhor valor já encontrado representado pela variável bestvalue, o ramo é cortado. Assim, o algoritmo evita a análise de combinações inviáveis ou com pouco potencial, tornando a busca muito mais eficaz do que uma abordagem puramente exaustiva como a do backtracking, além de garantir a corretude da solução ótima encontrada.

3.2.1. Análise de Corretude

A corretude de um algoritmo consiste em garantir, por meio de prova matemática, que em qualquer execução cada bloco realiza exatamente o que se espera; para isso, é necessário identificar claramente o objetivo de cada bloco, determinar o estado do sistema antes de sua execução, analisar o efeito que esse bloco produz sobre esse estado e, por fim, caracterizar o estado resultante após sua execução, assegurando que ele esteja de acordo com o comportamento esperado do algoritmo.

Para a análise de Corretude do algoritmo implementado utilizando a estratégia de Branch and Bound, devemos definir uma invariante de laço e provar inicialização, manutenção e término.

Invariante de laço:

A cada chamada do método branchAndBound, a lista currentTasks representa uma solução parcial válida (respeitando os deadlines), e bestValue contém o maior valor entre todas as soluções completas já exploradas até o momento.

Inicialização:

Antes que ocorra a primeira chamada do algoritmo, o elemento currentTasks está vazio, ou seja, é uma solução válida. Ainda nesse momento inicial, bestValue vale zero, o que é coerente e correto já que nenhuma solução foi explorada no momento. Assim, a invariante é verdadeira no início do algoritmo.

Manutenção:

Para cada chamada do algoritmo recursivo, a validade da solução parcial é mantida, já que só são incluídas as tarefas que respeitam o deadline. Os valores de currentTask e currentValue continuam coerentes, e bestValue só é atualizado quando uma solução melhor e completa é identificada. Soluções ótimas não são removidas por poda, já que são descartados soluções cujo valor máximo possível não supera o valor de bestValue. Assim, a invariante permanece verdadeira a cada passo do algoritmo.

Término:

No fim de todas as chamadas recursivas, foram podadas ou exploradas corretamente todas as soluções viáveis. Pela invariante, temos que bestValue possui o maior valor possível e bestSolution corresponde a uma solução ótima do problema.

3.2.2. Análise de Desempenho

O desempenho do algoritmo Branch and Bound depende da estruturação do problema e da eficiência da função de poda utilizada. No pior caso, o algoritmo irá se comportar como uma busca exaustiva, já que buscará explorar, assim como no backtracking, todas as combinações viáveis de soluções. Porém, essa estratégia possui sua vantagem única de eliminar ramos não promissores da árvore de busca através do uso dos bounds (limites). Essa vantagem garante uma redução significativa do número de estados explorados na prática.

O desempenho do algoritmo pode ser analisado considerando o número de nós explorados durante a execução do código, essa métrica é representada pela variável nodesExplored. Com esse elemento, é possível avaliar empiricamente o impacto da utilização da poda, sendo evidenciado que, embora o limite superior seja elevado, o custo real geralmente tende a ser menor em instâncias em que a função de bound é eficiente. Portanto, o código, dependendo da entrada, mescla um desempenho prático otimizado com um pior caso exponencial.

O Branch and Bound é um algoritmo de resultado exato, ou seja, ele sempre irá construir uma solução ótima do problema, e não uma solução aproximada. A qualidade da solução não é comprometida pelo uso da função que poda, pois apenas ramos que garantidamente não podem superar a melhor solução atual são eliminados. Assim, existe um equilíbrio entre qualidade e desempenho em que o código reduz o espaço de busca sem perder a garantia de ser ótimo.

Em relação ao consumo de memória do algoritmo, é apresentado um comportamento eficaz, já que é utilizado um espaço correspondente ao tamanho da entrada. A profundidade da recursão é restrita pela quantidade de tarefas, enquanto estruturas de apoio, como a lista de tarefas ativas (currentTasks) e a solução ótima (bestSolution), têm um limite de n elementos. Dado que o algoritmo não guarda todas as soluções viáveis na memória, mas sim apenas a solução atual e a melhor

que foi identificada, a utilização de espaço é linear, apresentando uma complexidade espacial de O(n).

3.2.2.1. Espacial

Nesse algoritmo, temos uma complexidade espacial linear, O(n). Sendo n a representação do número de tarefas, a profundidade máxima da recursão é, logicamente, proporcional ao tamanho da entrada, já que o algoritmo percorre as tarefas de forma sequencial.

É perceptível, também, que as estruturas auxiliares (currentTask e bestSolution) possuem um tamanho máximo de n. Já que o algoritmo não guarda todas as soluções possíveis, mas sim a melhor encontrada e a atual, o consumo de memória se torna bastante eficiente. Assim, concluímos que o espaço usado cresce de forma linear com o número total de tarefas.

3.2.2.2. Temporal

No pior caso, a complexidade temporal do algoritmo é exponencial, $O(2^n)$. Sendo n a representação do número de tarefas, sendo que para cada tarefa existem somente duas decisões válidas (incluir ou não incluir), consequentemente criando uma árvore de busca com 2^n nós. Porém, isso só ocorre quando o algoritmo não consegue chamar a função de poda que elimina ramos descartáveis da árvore, resultando em uma análise de todas as combinações possíveis da árvore.

Entretanto, na prática concreta, o cálculo do upperBound (limite superior) permite o uso da função de poda, aumentando o desempenho e otimizando a solução do problema. Essa estratégia permite a interrupção de ramos que não podem originar soluções superiores do que a já encontrada, eliminando chamadas recursivas desnecessárias. Porém, essa eficácia depende da qualidade da função de bound e dos dados da entrada passados para o algoritmo. Assim, não é possível definir uma complexidade temporal fechada para o caso médio, já que a análise depende da instância do problema.

3.3. Greedy

A estratégia de algoritmos conhecida como estratégia gulosa (Greedy) consiste em construir soluções através de decisões locais ótimas em cada etapa do processo de solução. Essa técnica se difere das anteriores que exploram diversas

combinações possíveis para o alcance da melhor solução global, já que o algoritmo guloso seleciona, a cada passo, a alternativa que aparentemente é mais vantajosa naquele momento da execução, sem reconsiderar decisões passadas.

O funcionamento desse método guloso baseia-se na ordenação ou avaliação dos elementos do problema seguindo um critério de prioridade (valor) definido previamente. Posteriormente, o algoritmo segue percorrendo a estrutura escolhendo iterativamente os elementos que satisfazem de forma ideal esse critério, porém, não podem violar as restrições impostas pelo problema. O processo citado perdura até que não seja mais viável incluir novos elementos ou até que todos tenham sido verificados.

Para o problema de escalonamento de tarefas, em específico, a estratégia gulosa foi integrada considerando o relacionamento entre o valor da tarefa e o seu tempo de processamento. Dessa forma, inicialmente as tarefas são ordenadas em ordem decrescente da razão valor/tempo (value/processingTime) de processamento, priorizando as tarefas que oferecem maior retorno por unidade de tempo. Após o término dessa ordenação, a lista de tarefas será percorrida, sendo o algoritmo responsável por avaliar a viabilidade de inserção das tarefas respeitando o prazo de execução (deadline). Caso a tarefa possa ser executada sem violar o prazo, ela será adicionada à solução e seu tempo de processamento será somado ao tempo total acumulado pelas tarefas escolhidas.

Esse método de solução possibilita o encontro rápido de uma solução viável para o problema de escalonamento, porém não existe garantia de que a solução obtida seja obrigatoriamente a ótima.

3.3.1. Análise de Corretude

Para o algoritmo guloso implementado implementado para, em específico, o problema de escalonamento de tarefas, a corretude está associada à garantia de que todas as tarefas inseridas na solução final respeitam suas correspondentes deadlines (prazos).

Invariante de laço:

A cada iteração do laço que percorre a lista de tasks ordenadas, a lista de tasks selecionadas representa uma solução parcial válida, ou seja, todas as tasks incluídas até esse momento respeitam seus correspondentes deadlines, e o tempo

total acumulado corresponde de forma correta à soma dos tempos de processamento dessas tasks.

Inicialização:

Antes da primeira iteração do algoritmo guloso, nenhuma task foi selecionada e o tempo total acumulado é zero. Assim, a lista de tasks selecionadas está vazia, representando uma solução válida para o problema, pois nenhuma deadline foi violada. Assim, concluímos que a invariante de laço é verdadeira antes do início da execução do algoritmo.

Manutenção:

Durante cada iteração do algoritmo, é analisada uma task da lista ordenada. Antes de a colocar na solução, é averiguado se a soma do tempo de processamento acumulado com o tempo da task candidata permanece menor ou igual ao seu deadline. Caso essa condição seja satisfeita, a task é adicionada à lista de tarefas selecionadas da solução e o tempo total é, também, atualizado. Caso contrário, a task é descartada. Em ambos os casos, a solução parcial continua válida, pois nenhuma task incluída viola seu prazo de execução. Assim, a invariante permanece verdadeira ao longo de todas as iterações do algoritmo.

Término:

Ao final da execução do laço, todas as task da lista ordenada foram averiguadas. Como a invariante de laço garante que todas as tasks incluídas respeitem seus deadlines correspondentes, a solução final corresponde a um subconjunto válido de tasks executáveis dentro de seus deadlines. Assim, a estratégia do algoritmo produz uma solução válida para o problema de escalonamento de tarefas. Entretanto, por se tratar de uma estratégia gulosa baseada em decisões locais ótimas, não existe garantia de que a solução encontrada seja a ótima para todas as instâncias do problema analisado.

3.3.2. Análise de Desempenho

O algoritmo guloso é, surpreendente, superior em questão de desempenho às outras abordagens exaustivas e a dinâmica. Isso é consequência do algoritmo não explorar todas as combinações possíveis das tasks, voltando em estruturas

complexas como árvores, mas sim realizando apenas uma única passagem pela lista de tasks que foi previamente ordenada.

O principal custo computacional deste método está na fase de ordenação das tarefas, já que ela é realizada com base na razão entre valor e tempo de processamento de cada task, o que permite, também, priorizar as tarefas que promovem um maior benefício.

Posteriormente à ordenação, a estratégia gulosa percorre a lista outra vez, certificando que cada task pode ser inserida na solução atual sem violações de deadlines. Pelo fato de não existirem chamadas recursivas em exploração de múltiplos ramos da árvore, as operações executadas acabam reduzindo consideravelmente.

Em muitos casos práticos, em termos de qualidade da solução, esse algoritmo pode gerar resultados próximos da solução ótima. Porém, o algoritmo baseia-se somente em decisões locais, ou seja, não há garantia de que o resultado obtido seja de fato o melhor possível para todas as instâncias do problema. Isso é perceptível ao realizar chamadas exaustivas do método para diferentes instâncias, resultando em outputs incorretos em relação a resposta ótima do problema. Ainda assim, a estratégia possui um excelente balanceamento entre qualidade da solução e custo computacional, principalmente nos problemas com um grande número de tarefas.

3.3.2.1. Espacial

A complexidade espacial do algoritmo de estratégia gulosa é linear, representada por O(n) (n corresponde ao número de tasks na entrada do problema).

Essa complexidade é justificada pelo fato do algoritmo utilizar somente algumas estruturas de auxílio simples durante toda a execução. Essas estruturas são a lista que armazena as tasks selecionadas durante a execução (selectedTasks) e a lista de tasks ordenadas (sortedTasks). Já que não é utilizada nenhuma estrutura adicional relativamente complexa, o algoritmo não mantém na memória todas as combinações viáveis de tasks, sendo que o consumo de memória aumenta proporcionalmente ao tamanho da entrada do problema. Assim, é perceptível que o algoritmo apresenta um uso eficiente de memória em relação às outras abordagens de exploração complexa.

3.3.2.2. Temporal

A etapa de ordenação das tasks domina a complexidade temporal do algoritmo de estratégia gulosa. Considerando que essa ordenação é realizada pelo algoritmo de ordenação interno do método sort da linguagem Java, o custo dessa fase do algoritmo é da ordem de O(n log n).

Com o fim da ordenação, a lista de tasks é percorrida mais uma vez, verificando se cada elemento pode ser inserido na solução atual sem violar os prazos (deadline). Essa etapa possui um custo linear, esse representado por O(n). Assim, concluímos que a complexidade temporal total é dada por: O(n log n).

Essa complexidade é consideravelmente menor que a complexidade exponencial ilustrada pelos algoritmos de branch and bound e backtracking no pior caso. Isso reforça ainda mais a ideia da estratégia gulosa ser bastante eficiente para instâncias de grande tamanho. Porém, é sempre importante ressaltar que esse alto desempenho possui um preço, sendo ele a não garantia de uma solução ótima para todas as instâncias do problema proposto.

3.4. Dynamic Programming

A técnica de Programação Dinâmica é aplicada a problemas de otimização que apresentam subestrutura ótima e sobreposição de subproblemas. Sua abordagem consiste em decompor o problema original em subproblemas menores, armazenando suas soluções para evitar cálculos repetitivos.

No problema de escalonamento de tarefas, cada tarefa é definida por um tempo de processamento, um valor e um prazo. O objetivo é selecionar um subconjunto de tarefas que maximize o valor total, respeitando as restrições de tempo impostas pelos deadlines.

A implementação utiliza um vetor de estados dp, no qual cada posição dp[t] representa o maior valor possível obtido ao se utilizar um tempo total t. A solução é construída de forma incremental: para cada tarefa, verifica-se se sua inclusão em determinado estado de tempo resulta em melhoria do valor previamente armazenado.

A característica de subestrutura ótima é observada no fato de que a melhor solução para um tempo t depende diretamente da melhor solução previamente calculada para o tempo t - processingTime. Já a sobreposição de subproblemas

ocorre porque diferentes combinações de tarefas levam à avaliação repetida dos mesmos estados de tempo, os quais são armazenados e reutilizados no vetor dp.

O algoritmo percorre os estados de tempo de forma decrescente, garantindo que cada tarefa seja considerada apenas uma vez em cada estado, mantendo a consistência dos resultados. Ao final, o maior valor presente no vetor dp corresponde à solução ótima do problema. A estrutura auxiliar utilizada permite reconstruir o conjunto de tarefas selecionadas.

3.4.1. Análise de Corretude

A corretude do algoritmo de Programação Dinâmica baseia-se na garantia de que o vetor dp armazena, para cada tempo t, o maior valor possível obtido por um subconjunto de tarefas que respeita os deadlines. A prova é feita por meio de uma invariante de laço.

Invariante de Laço:

A cada iteração do algoritmo, para todo tempo t, o vetor dp[t] armazena o maior valor possível que pode ser obtido utilizando um tempo total igual a t, considerando apenas as tarefas já processadas e respeitando os deadlines.

Inicialização:

Antes do início da execução, o vetor dp é inicializado com zero em todas as posições. Isso representa corretamente o fato de que, sem considerar nenhuma tarefa, o valor máximo obtido é zero para qualquer tempo. Assim, a invariante é verdadeira no início do algoritmo.

Manutenção:

A cada nova tarefa considerada, o algoritmo percorre os estados de tempo e avalia a possibilidade de incluí-la. A atualização de dp[t] ocorre apenas quando a inclusão da tarefa respeita seu deadline e resulta em um valor superior ao já armazenado. Como o algoritmo sempre mantém o maior valor possível para cada tempo e utiliza apenas informações previamente calculadas, a validade da invariante é preservada a cada iteração.

Término:

Ao final do processamento de todas as tarefas, o vetor dp contém os melhores valores possíveis para todos os tempos viáveis. O maior valor armazenado em dp corresponde à solução ótima do problema, e a estrutura auxiliar permite recuperar o conjunto de tarefas associado a esse valor. Dessa forma, o algoritmo garante a corretude da solução produzida.

3.4.2. Análise de Desempenho

O desempenho do algoritmo de Programação Dinâmica está diretamente relacionado à forma como os estados são definidos e atualizados ao longo da execução. No código implementado, o problema é resolvido por meio de um vetor dp, que armazena os melhores valores possíveis para cada tempo, e de uma estrutura auxiliar take, utilizada para reconstruir a solução. Dessa forma, o custo do algoritmo depende tanto do número de tarefas quanto do limite máximo de tempo considerado, definido pelo maior deadline.

3.4.2.1. Temporal

A complexidade temporal do algoritmo é O(n × D), onde nnn representa o número de tarefas e DDD o maior deadline. Isso ocorre porque o algoritmo possui dois laços principais: o primeiro percorre todas as tarefas, enquanto o segundo percorre os possíveis estados de tempo, variando de DDD até o tempo de processamento da tarefa.

Para cada par tarefa-tempo, são realizadas apenas operações constantes, como comparações e atualizações no vetor dp. Assim, o tempo total de execução é proporcional ao produto entre o número de tarefas e o número de estados de tempo avaliados.

3.4.2.2. Espacial

A complexidade espacial do algoritmo também é O(n × D). No código, isso se deve à utilização de duas estruturas principais: o vetor dp, de tamanho proporcional a D, e a matriz take, com dimensão n × D, responsável por armazenar as decisões tomadas durante o processamento.

Enquanto o vetor dp mantém os valores ótimos intermediários, a matriz take registra quais tarefas foram incluídas em cada estado, permitindo a reconstrução da

solução ao final do algoritmo. Dessa forma, o consumo de memória cresce proporcionalmente ao número de tarefas e ao maior tempo considerado.

4.EXPERIMENTOS E RESULTADOS

4.1. Instâncias de Teste

Para a execução dos experimentos, foram geradas cinco instâncias com 5, 10, 15, 20 e 25 tarefas, cada uma contendo informações sobre tempo de processamento, prazo e valor de cada tarefa. Essas instâncias são armazenadas no arquivo input.txt dentro da pasta instances. O sistema lê esse arquivo, transforma os dados em objetos da classe Task e organiza cada conjunto de tarefas para processamento pelos algoritmos implementados. Essa estrutura permite que todo o fluxo (geração, leitura e preparação das instâncias) ocorra de forma automática, garantindo que cada instância seja interpretada corretamente pelo sistema.

Para cada instância lida, os algoritmos Backtracking, Branch and Bound, Programação Dinâmica e Estratégia Gulosa são aplicados de forma independente. Cada algoritmo é executado quinze vezes por instância, com o objetivo de medir o tempo médio de execução e reduzir a influência de variações pontuais. Durante cada repetição, o tempo de processamento é registrado em milissegundos e a solução encontrada é armazenada como um objeto da classe Solution. Após as quinze execuções, é calculada a média dos tempos e as soluções correspondentes são adicionadas a listas específicas para cada algoritmo, garantindo que os resultados considerados na análise final sejam consistentes e representativos.

Ao final da execução, todas as soluções e os tempos médios obtidos são salvos em um arquivo de saída chamado output.txt, localizado na pasta results. Esse registro permite comparar diretamente o desempenho de Backtracking e Branch and Bound para cada instância, além de possibilitar a reprodução completa dos experimentos. Dessa forma, o fluxo do programa integra geração das instâncias em input.txt, processamento pelos algoritmos, cálculo de médias e armazenamento em output.txt.

4.2. Resultados Obtidos

4.2.1. Configurações do Ambiente

Tabela 04: Ambiente de Testes

Processador	Intel(R) Core(TM) i3-1005G1 CPU @ 1.20GHz (1.19 GHz)
Memória RAM	20,0 GB
Sistema Operacional	Microsoft Windows 11 Home Single Language
JVM	openjdk version 24.0.1
Outros detalhes	Nenhum outro processo relevante estava sendo executado durante a execução dos testes, a fim de minimizar interferências no desempenho dos algoritmos.

Fonte: elaborada pelos autores

4.2.2. Análise dos Resultados

Tabela 05: Tempos Médios de Execução por Algoritmo e Instância

Instâncias	Tamanho	Algoritmo	Tempo médio (ms)
Instância 1	5	Backtracking	0,057 ms
		Branch and Bound	0,048 ms
		Greedy	0,046 ms
		Dynamic Programming	0,438 ms
Instância 2	10	Backtracking	2,086 ms
		Branch and Bound	0,592 ms
		Greedy	0,099 ms
		Dynamic Programming	0,153 ms
Instância 3	15	Backtracking	0,209 ms
		Branch and Bound	0,114 ms
		Greedy	0,020 ms
		Dynamic Programming	0,232 ms
Instância 4	20	Backtracking	0,620 ms
		Branch and Bound	0,095 ms
		Greedy	0,076 ms
		Dynamic Programming	0,151 ms
Instância 5	25	Backtracking	4,021 ms

		Branch and Bound	0,114 ms
		Greedy	0,017 ms
		Dynamic Programming	0,133 ms

Fonte: elaborada pelos autores

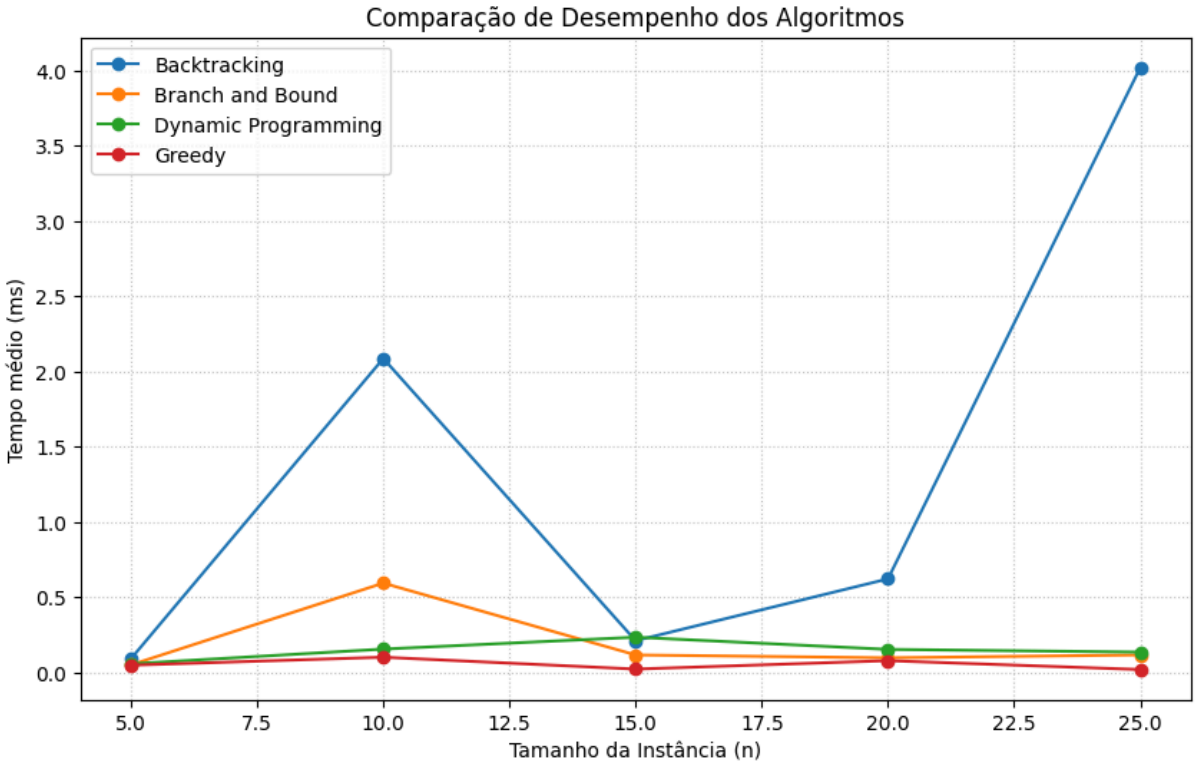
Tabela 06: Nós explorados da árvore por Algoritmo e Instância

Instâncias	Tamanho	Algoritmo	Nós explorados
Instância 1	5	Backtracking	63
		Branch and Bound	11
Instância 2	10	Backtracking	832
		Branch and Bound	53
Instância 3	15	Backtracking	21719
		Branch and Bound	634
Instância 4	20	Backtracking	36998
		Branch and Bound	1001
Instância 5	25	Backtracking	780685
		Branch and Bound	11198

Fonte: elaborada pelos autores

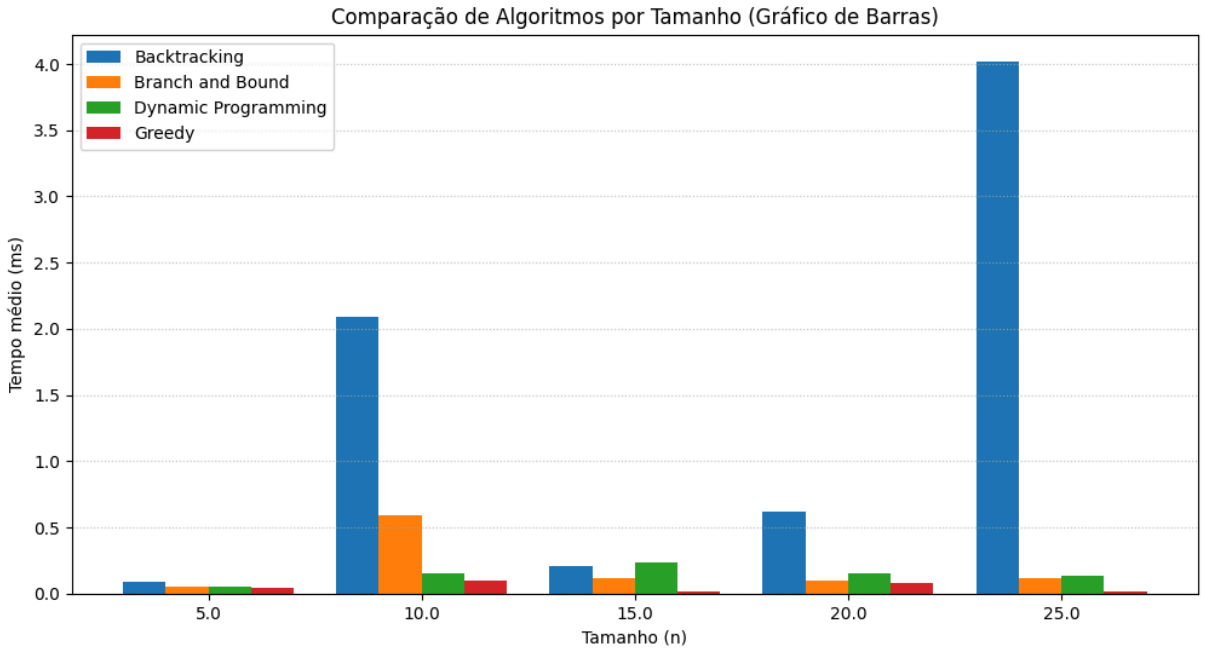
Gráficos:

Gráfico 01: Gráfico de Linhas dos Tempos Médios por Instância



Fonte: elaborada pelos autores

Gráfico 02: Gráfico de Barras dos Tempos Médios por Instância



Fonte: elaborada pelos autores

Em ambiente controlado, conforme descrito anteriormente, foram realizados os testes de execução dos algoritmos, resultando nos valores apresentados na

Tabela 05. Os Gráficos 01 e 02 ilustram visualmente os tempos médios obtidos para cada algoritmo em função do tamanho das instâncias, permitindo observar a variação desses tempos ao longo dos experimentos realizados.

Complementarmente, a Tabela 06 apresenta a quantidade de nós explorados na árvore de busca por cada algoritmo em cada uma das instâncias consideradas, sendo possível comparar, por meio dessas estruturas, as diferenças no comportamento de exploração entre os algoritmos.

5. ANÁLISE E DISCUSSÃO

5.1. Comparação das Estratégias

Na Tabela 05, é possível analisar empiricamente o tempo de execução dos algoritmos considerados para diferentes tamanhos de instâncias. Esse comportamento também é evidenciado nos Gráficos 01 e 02, apresentados, respectivamente, nos formatos de linhas e barras, os quais permitem uma visualização comparativa mais clara da evolução dos tempos médios. A partir desses resultados, observa-se uma diferença significativa no comportamento dos algoritmos tanto em termos de custo computacional quanto de desempenho prático.

Os algoritmos Backtracking e Branch and Bound, ambos baseados na exploração do espaço de busca, apresentam custo exponencial no pior caso. No entanto, o Branch and Bound demonstra desempenho consistentemente superior ao Backtracking em todas as instâncias analisadas. Isso se deve ao uso de estratégias de poda, que reduzem significativamente a quantidade de soluções exploradas. Em contrapartida, o Backtracking realiza uma busca exaustiva, o que resulta em um crescimento acentuado do tempo de execução à medida que o tamanho da instância aumenta.

O algoritmo de Programação Dinâmica, por sua vez, apresenta um custo inferior em relação aos métodos de busca, uma vez que evita cálculos repetitivos por meio do armazenamento de subproblemas já resolvidos. Esse comportamento resulta em tempos de execução mais estáveis e, em geral, mais eficientes que o Backtracking, embora ainda superiores aos observados para o algoritmo guloso em algumas instâncias. Além disso, assim como os algoritmos baseados em busca, a Programação Dinâmica garante a obtenção da solução ótima, mantendo a corretude dos resultados.

Por outro lado, o algoritmo Greedy apresentou o melhor desempenho em termos de tempo de execução, sendo o mais rápido em todas as instâncias analisadas. Esse resultado é esperado, visto que sua abordagem constrói a solução com base em decisões locais ótimas, sem a necessidade de explorar o espaço completo de soluções, o que reduz significativamente o custo computacional.

Entretanto, essa eficiência vem acompanhada de uma limitação importante relacionada à corretude das soluções. Durante os experimentos, observou-se que o algoritmo guloso apresentou inconsistências nos outputs, indicando que nem sempre é capaz de encontrar a solução ótima. Esse comportamento caracteriza o principal trade-off dessa abordagem que embora seja extremamente eficiente, não há garantia de resultado ótimo, diferentemente dos algoritmos Backtracking, Branch and Bound e Programação Dinâmica, que sempre produzem soluções ótimas.

No que se refere à Tabela 06, observa-se a quantidade de nós explorados na árvore de busca por cada algoritmo em função do tamanho das instâncias. Nota-se um crescimento expressivo no número de nós explorados pelo Backtracking, especialmente nas instâncias de maior porte, evidenciando o impacto de sua abordagem exaustiva. Em contrapartida, o Branch and Bound apresenta uma redução significativa nesse número, reforçando a eficácia de suas estratégias de poda.

De modo geral, os resultados evidenciam que a escolha do algoritmo envolve um equilíbrio entre desempenho e corretude. Enquanto algoritmos exatos garantem soluções ótimas com maior custo computacional, abordagens heurísticas como o Greedy oferecem maior eficiência, porém sem garantia de soluções ótimas.

6.CONCLUSÃO

Os resultados obtidos evidenciam diferenças relevantes no desempenho prático entre os algoritmos analisados. Observou-se que o Branch and Bound apresenta maior eficiência na maioria das instâncias, especialmente com o aumento do tamanho da entrada, em decorrência da redução do espaço de busca proporcionada pelo mecanismo de poda. Em contrapartida, o Backtracking explora um número significativamente maior de soluções, o que impacta diretamente no tempo de execução.

Nesse contexto, torna-se evidente que não apenas o custo assintótico, mas também as estratégias adotadas na exploração do problema exercem influência determinante no desempenho observado. A eficiência do Branch and Bound reforça a importância de técnicas que reduzam o espaço de busca, enquanto a Programação Dinâmica se destaca por sua capacidade de reutilizar resultados intermediários, promovendo maior estabilidade no tempo de execução.

Por sua vez, o algoritmo guloso evidencia que abordagens heurísticas podem alcançar desempenho significativamente superior em termos de tempo, ainda que à custa da qualidade das soluções em determinados casos. Tal comportamento reforça a necessidade de avaliar cuidadosamente os requisitos do problema antes da escolha da abordagem, considerando limitações e objetivos específicos.

Dessa forma, conclui-se que a escolha do algoritmo mais adequado deve considerar não apenas o desempenho observado, mas também as características específicas do problema a ser resolvido. No contexto deste trabalho, a Programação Dinâmica mostrou-se a alternativa mais apropriada, ao equilibrar eficiência computacional e garantia de otimalidade. Contudo, é fundamental interpretar as particularidades de cada problema, uma vez que diferentes cenários podem exigir abordagens distintas, seja priorizando a qualidade da solução ou o tempo de execução.

7.REFERÊNCIAS

CORMEN, T. H.; AL, E. Algoritmos : teoria e prática. Rio De Janeiro: Elsevier, 2002. Acesso em 07 mar. 2026

DECHTER, R.; FROST, D. Backjump-based backtracking for constraint satisfaction problems. Artificial Intelligence, v. 136, n. 2, p. 147–188, abr. 2002. Acesso em 10 mar. 2026